Chapter 7
Stochastic Optimization
Lectures on Monte Carlo Theory

Leskelä & Vihola

Chapter Outline

The Stochastic Optimization Problem

Goal. Given a function $h : \mathcal{S} \to \mathbb{R}$, find

Optimization Objective

$$\gamma^{\mathrm{opt}} = \max_{x \in \mathcal{S}} h(x), \qquad x^{\mathrm{opt}} = \arg\max_{x \in \mathcal{S}} h(x).$$

- $\mathcal{S}$ – state / solution space (may be combinatorial)
- h – objective / performance function (to be *maximized*)
- Equivalently minimize cost $H = -h$

Core idea (Monte Carlo approach): construct a Markov chain whose stationary distribution π puts high probability on good solutions, then sample from it.

Methods covered:

1. Metropolis / Gibbs at fixed temperature
2. Simulated Annealing (decreasing temperature)
3. Locally Informed Proposals (gradient-aware)
4. Cross-Entropy (CE) method
5. Metropolis–CE hybrid

Running Examples Throughout the Chapter

Two combinatorial problems are used as benchmarks:

Travelling Salesman Problem (TSP)

- n cities, symmetric distance matrix $\mathbf{M}$
- Find permutation $\sigma^* \in S_n$ minimising total distance

$$l(\sigma) = \sum_{k=1}^{n-1} \mathbf{M}(\sigma(k), \sigma(k+1)) + \mathbf{M}(\sigma(n), \sigma(1))$$

- NP-hard; optimal for $n = 13$ is **7024** (Held-Karp)

0-1 Knapsack Problem

- d items, weights $\mathbf{w}$, values $\mathbf{v}$, capacity W
- Find $\mathbf{x} \in \{0,1\}^d$ maximising $\mathbf{v} \cdot \mathbf{x}$ subject to $\mathbf{w} \cdot \mathbf{x} \leq W$
- Simulation: $d = 100$, $w_i = i$, $v_i = i^{1.2}$, $W = 3000$

7.1 Metropolis and Gibbs Based Methods

Setting. Graph $G = (V, E)$, cost $H : V \to \mathbb{R}_+$. Want to find $\arg\min_v H(v)$.

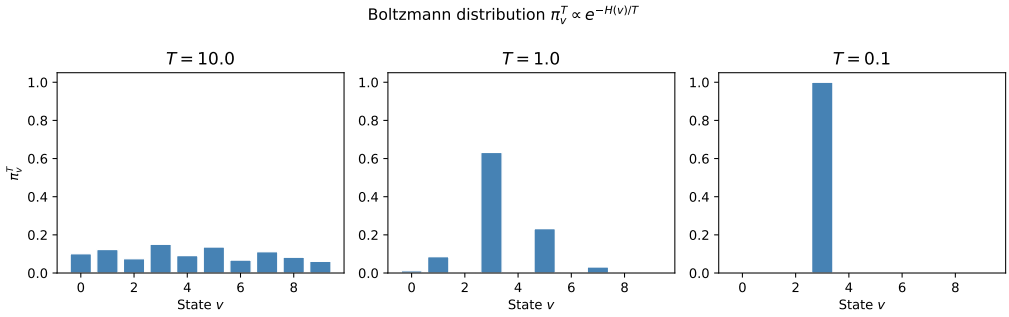
Boltzmann Distribution (Eq. 7.1.1)

$$\pi_v^T = \frac{e^{-H(v)/T}}{\sum_{v' \in V} e^{-H(v')/T}}, \qquad v \in V, \quad T > 0.$$

- T – *temperature* parameter
- As $T \to 0$: mass concentrates on minimisers of H
- As $T \to \infty$: approaches uniform distribution
- Key identity: $\arg\min_v H(v) = \arg\max_v \pi_v^T$

To maximise $f : V \to \mathbb{R}_+$, set $H(v) = -f(v)$.

Boltzmann Distribution at Different Temperatures



As T decreases, the Boltzmann distribution concentrates mass on the minimiser of H (the state with smallest cost).

Acceptance Probability for Metropolis on Graph

Use the **random walk** on G as proposal matrix $\mathbf{Q}$. The Metropolis acceptance probability is:

Acceptance Probability (Algorithm 61)

$$\alpha_{vv'}^{T} = \min\left(1,\ \frac{\deg(v)}{\deg(v')}\, e^{(H(v)-H(v'))/T}\right), \qquad v, v' \in V.$$

$$p_{vv'}^{T} = \frac{1}{\deg(v)} \min\left(1,\ \frac{\deg(v)}{\deg(v')}\, e^{(H(v)-H(v'))/T}\right), \quad q_{vv'} > 0.$$

- $\deg(v)$ – degree of vertex v in G
- Moves to a *worse* solution ($H(v') > H(v)$) with positive prob
- This allows escape from local minima

Algorithm 61: Metropolis for Boltzmann Distribution

```python
import numpy as np

def metropolis_boltzmann_step(v, neighbors, H, T, rng):
    """One step of Metropolis-Hastings for Boltzmann distribution."""
    deg_v  = len(neighbors[v])
    v_prop = rng.choice(neighbors[v])              # propose neighbor uniformly
    deg_vp = len(neighbors[v_prop])
    # Acceptance probability
    alpha = min(1.0,
                (deg_v / deg_vp) * np.exp((H[v] - H[v_prop]) / T))
    U = rng.uniform()
    return v_prop if U <= alpha else v

# Run for n_steps
def run_metropolis(v0, neighbors, H, T, n_steps, seed=0):
    rng   = np.random.RandomState(seed)
    chain = [v0]
    v     = v0
    for _ in range(n_steps):
        v = metropolis_boltzmann_step(v, neighbors, H, T, rng)
        chain.append(v)
    return chain
```

Application: Decrypting a Substitution Cipher

Setup. A substitution cipher permutes the alphabet: $\sigma : \Sigma \to \Sigma$, $|\Sigma| = 26$. State space $\mathcal{S} = S_{26}$ ($26! \approx 4 \times 10^{26}$ permutations).

Log-Likelihood Score Function

$$l(\sigma) = \sum_{i=1}^{L-1} \log \mathbf{M}(\sigma(c_i), \sigma(c_{i+1})),$$

where $\mathbf{M}(a, b)$ is the bigram frequency in English text (*War and Peace*), and $c_1, \ldots, c_L$ is the ciphertext.

Boltzmann distribution: $\pi_\sigma = \frac{1}{z} l(\sigma)$ (proportional to likelihood).
Proposal: swap two uniformly chosen symbols (transposition).

Result: After 3000 steps the message is essentially decrypted (log-likelihood improves from -4274 to -815).

Metropolis Cipher Decryption: Log-Likelihood Trace

Fig 7.1 – Metropolis for substitution cipher: log-likelihood

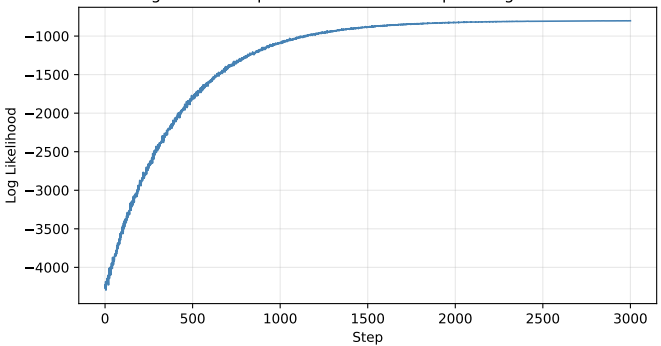


Table 7.1 (selected steps):

Step	Decrypted (start)	Log-lik
1	yckqzchskqhaqyc...	−4274
1000	the whole of that...	−896
2000	...anna skent...	−826
3000	...anna spent...	−815

Rapid improvement in first 500 steps, then gradual refinement.

The 0-1 Knapsack Problem

Setup. d items, weights $\mathbf{w} = (w_1, \ldots, w_d)$, values $\mathbf{v} = (v_1, \ldots, v_d)$. Find $\mathbf{x} \in \{0,1\}^d$ maximising total value subject to capacity W.

Boltzmann Distribution for Knapsack

$$\pi_{\mathbf{x}} = \begin{cases} \dfrac{\exp(\mathbf{v} \cdot \mathbf{x} / T)}{z_T} & \text{if } \mathbf{w} \cdot \mathbf{x} \leq W, \\ 0 & \text{otherwise.} \end{cases}$$

States violating the weight constraint have zero probability.

Gibbs conditional (Algorithm 63):

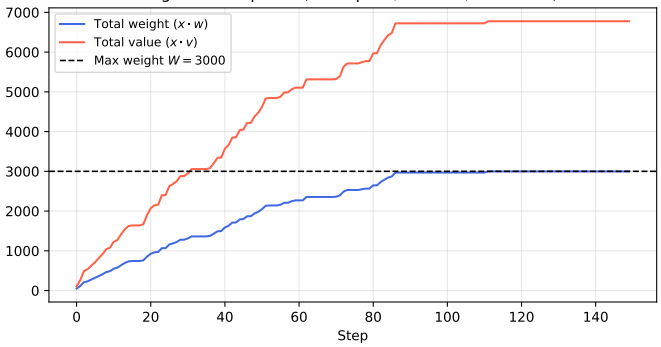
$$\mathbb{P}(Z(i) = 1 \mid Z_{-i} = \mathbf{x}_{-i}) = \frac{1}{1 + \exp\left(-\frac{1}{T} v_i x_i\right)}.$$

Algorithms 63 & 64: Gibbs and Metropolis for Knapsack

```python
import numpy as np

def gibbs_knapsack_step(x, w, v, W, T, rng):
    """Gibbs sampler step for 0-1 knapsack (Algorithm 63)."""
    d = len(x);  i = rng.randint(0, d)
    x_new = x.copy();  x_new[i] = 1
    if np.dot(w, x_new) > W:
        x_new[i] = 0           # forced: adding item exceeds capacity
    else:
        prob1 = 1.0 / (1.0 + np.exp(-v[i] / T))
        x_new[i] = 1 if rng.rand() <= prob1 else 0
    x[i] = x_new[i];  return x

def metropolis_knapsack_step(x, w, v, W, T, rng):
    """Metropolis step for 0-1 knapsack (Algorithm 64)."""
    d = len(x);  i = rng.randint(0, d)
    x_new = x.copy();  x_new[i] = 1 - x_new[i]    # flip bit i
    if np.dot(w, x_new) <= W:
        alpha = min(1.0, np.exp((1/T) * (1 - 2*x[i]) * v[i]))
    else:
        alpha = 0.0
    if rng.rand() < alpha:
        x[:] = x_new
    return x
```

Knapsack Simulation Results

Fig 7.2 – Knapsack (Metropolis, $d = 100$, $W = 3000$)



Parameters:
$d = 100$, $W = 3000$
$w_i = i$, $v_i = i^{1.2}$, $T = 1$

After 150 steps:

- Total weight $X_{150} \cdot \mathbf{w} \approx 2977$
- Total value $X_{150} \cdot \mathbf{v} \approx 6745$

The weight stays near capacity $W = 3000$ while value climbs steadily.

Remark: results improve by tuning T.

7.2 Simulated Annealing – Motivation

Problem with fixed temperature:

- T large $\Rightarrow$ moves freely, but stationary distribution nearly uniform (poor optimiser).
- T small $\Rightarrow$ stationary distribution near optimal, but chain mixes slowly (gets trapped in local minima).

Key idea (Simulated Annealing): Start with a high temperature T_0 and *decrease* it according to a **cooling schedule** $T_0 \geq T_1 \geq \cdots \searrow 0$.

Modified Boltzmann (per step k)

$$\pi_v^{T_k} = \frac{\deg(v)\exp(-H(v)/T_k)}{z_{T_k}}, \qquad v \in V.$$

The transition matrix at step k is

$$p_{vv'}(k) = \frac{1}{\deg(v)} \min\Big(1,\, \exp\Big(-\tfrac{1}{T_k}(H(v') - H(v))\Big)\Big).$$

Convergence of Simulated Annealing

Proposition 7.2.2 (Convergence at Fixed T)

For fixed $T_k \equiv T$ the non-homogeneous MC satisfies

$$\lim_{T \searrow 0} \lim_{\substack{k \to \infty \\ T_k = T}} \mathbb{P}(X_k \in D^*) = 1,$$

where $D^* = \{v \in V : H(v) = \min_{v'} H(v')\}$ is the optimal set.

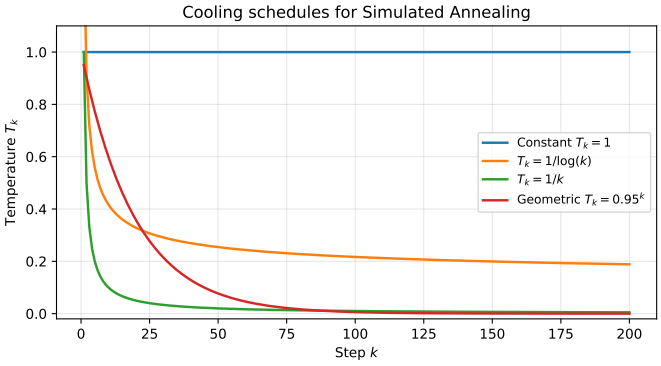
Cooling schedule constraint: Let $c = \min_{v' \in \mathcal{N}(v)}(H(v') - H(v))$ for a local minimum v. If

$$\sum_j e^{-c/T_j} < \infty$$

then the chain can *remain* in the local minimum – cooling is too fast.

Practical guidance: Use $T_k = d \log(k)$ for some $d > 0$ ("logarithmic cooling"). This is slow but theoretically justified.

Cooling Schedules



Common cooling schedules:

- **Constant**: $T_k = T$ (plain Metropolis)
- **Logarithmic**: $T_k = 1/\log(k)$
  Theoretically valid, but slow
- **Linear**: $T_k = 1/k$
  Too fast – can trap
- **Geometric**: $T_k = \beta^k$, $\beta < 1$
  Popular in practice

Convergence requires $T_k \searrow 0$ *slowly enough*.

Non-Homogeneous Markov Chains

Definition. A sequence (Y_n) is a **time non-homogeneous Markov chain** on $\mathcal{S} = \{s_1, \ldots, s_m\}$ if for any $s_{i_0}, \ldots, s_{i_k} \in \mathcal{S}$:

$$\mathbb{P}(Y_0 = s_{i_0}, Y_1 = s_{i_1}, \ldots, Y_k = s_{i_k}) = \mu_{s_{i_0}} p_{s_{i_0} s_{i_1}}(1) \cdots p_{s_{i_{k-1}} s_{i_k}}(k).$$

The transition probability at step k is $p_{s_i s_j}(k) = \mathbb{P}(Y_{k+1} = s_j \mid Y_k = s_i)$.

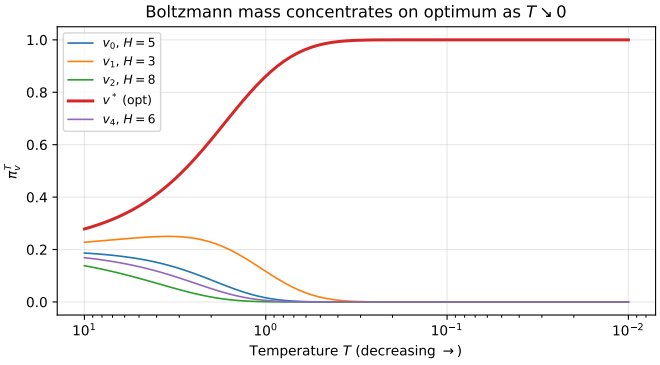
Distribution of Y_l (Eq. 7.2.3)

$$(\mathbb{P}(X_l = s_1), \ldots, \mathbb{P}(X_l = s_m)) = \boldsymbol{\mu}\, \mathbf{P}(0, l),$$

where $\mathbf{P}(k, l) = \mathbf{P}(k)\mathbf{P}(k+1) \cdots \mathbf{P}(l)$.

Remark 7.2.3. SA (Algorithm 65) is a non-homogeneous MC with $p_{vv'}(k) = q_{vv'}\, \alpha_k$.

Mass Concentration as $T \searrow 0$



As temperature decreases:

- All probability mass concentrates on the **optimal state** $v^* = \arg\min H$
- States with higher cost H lose mass exponentially fast
- Proof: denominator in π_v^T dominated by optimal terms

This justifies SA: run at decreasing temperatures to track the mass toward the global optimum.

Algorithm 65: Simulated Annealing (One Step)

```
def sa_step(v, neighbors, H, T_k, rng):
    """One SA step with temperature T_k (Algorithm 65)."""
    v_prop = rng.choice(neighbors[v])
    alpha_k = min(1.0, np.exp(-(H[v_prop] - H[v]) / T_k))
    U = rng.uniform()
    return v_prop if U <= alpha_k else v

def simulated_annealing(v0, neighbors, H, cooling, n_steps, seed=0):
    """Run SA with a given cooling schedule."""
    rng = np.random.RandomState(seed)
    v   = v0
    traj = [v]
    for k in range(n_steps):
        T_k = cooling(k + 1)          # temperature at step k+1
        v   = sa_step(v, neighbors, H, T_k, rng)
        traj.append(v)
    return traj

# Logarithmic cooling
def log_cooling(k, d=1.0):
    return d / np.log(k + 1)
```

SA for the Travelling Salesman Problem

Proposal: pick two distinct indices $i, j \in \{1, \dots, n\}$ uniformly at random; swap $\sigma(i)$ and $\sigma(j)$ (transposition).

Algorithm 67: SA for TSP

$$p_{\sigma\sigma'}(k) = \frac{2}{n(n-1)} \min\Big(1,\ e^{(l(\sigma)-l(\sigma'))/T_k}\Big)$$

where σ' differs from σ by one transposition.

2-opt variant (Remark 7.2.4): reverse a sub-tour $\sigma(i), \sigma(i-1), \dots, \sigma(j)$ – often faster in practice.

Cooling: $T_k = 1/\log(k)$ for 100 steps.

SA finds better solutions faster than plain Metropolis (60% of 10-step runs with SA beat plain Metropolis).

7.3 Locally Informed Proposals (LIP)

Limitation of uninformed Metropolis: Proposals are chosen *uniformly* among all neighbors – ignores information about which direction improves h.

Idea: let the target distribution π *guide* the proposal.

Locally Informed Proposal Distribution

Given current state s, sample a candidate $s^{(m)}$ from neighbors with probability proportional to the target:

$$\mathbf{q}_s^{\mathrm{LIP}} \propto \left(g\left(\frac{\pi_{s^{(1)}}}{\pi_s}\right) q_{ss^{(1)}},\ \ldots,\ g\left(\frac{\pi_{s^{(M)}}}{\pi_s}\right) q_{ss^{(M)}} \right),$$

for a *balancing function* $g(\cdot)$ (e.g. $g(r) = \sqrt{r}$).

The acceptance ratio then corrects for the biased proposal via

$$\alpha = \min\left(1,\ \frac{\pi_X\, q_{Xs}^{\mathrm{LIP}}}{\pi_s\, q_{sX}^{\mathrm{LIP}}}\right).$$

Algorithm 68: Locally Informed Metropolis-Hastings

```
def lip_step(s, pi, Q_neighbors, M_frac, g, rng):
    """
    One step of LIP (Algorithm 68).
    s: current state
    pi: stationary distribution (dict or array)
    Q_neighbors: {s: [(s', q_ss')] ...}
    M_frac: fraction of neighbors to subsample (beta)
    g: balancing function, e.g. lambda r: np.sqrt(r)
    """
    nbrs = Q_neighbors[s]
    # Subsample M candidates
    M = max(1, int(M_frac * len(nbrs)))
    idx = rng.choice(len(nbrs), M, replace=False)
    cands = [nbrs[i] for i in idx]

    # LIP weights
    weights = np.array([g(pi[sp] / pi[s]) * qss for sp, qss in cands])
    Z_g = weights.sum()
    probs = weights / Z_g
    # Draw candidate
    j = rng.choice(M, p=probs)
    X, q_sX = cands[j]
    # LIP probability of X -> s
    nbrs_X = Q_neighbors[X]
    wts_Xs = [g(pi[sp2] / pi[X]) * q for sp2, q in nbrs_X]
    q_LIP_Xs = g(pi[s] / pi[X]) * (sum(w for s2,_ in nbrs_X
                                   for w in [1]) / sum(wts_Xs))
```

LIP: Efficiency Improvements

Key efficiency observation:

- Standard Metropolis: evaluate ratio $\pi_{s'}/\pi_s$ for *one* randomly chosen neighbor s'.
- Full LIP: evaluate $g(\pi_{s'}/\pi_s)q_{ss'}$ for **all** $\binom{n}{2}$ neighbors – expensive!

Partial LIP (Fraction $\beta \in (0, 1]$)

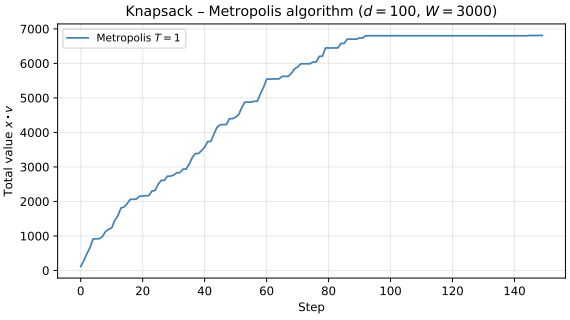
Sample $M = \lceil \beta\, \mathfrak{n} \rceil$ candidate neighbors (where $\mathfrak{n}$ is the total neighbor count), then

$$q_{ss(j)}^{\mathrm{LIP}} = \frac{1}{Z_g} g\left(\frac{\pi_{s(j)}}{\pi_s}\right) q_{ss(j)}.$$

For $\beta = 1$: full LIP. For small β: fast approximation.

Trade-off: $\beta = 1$ gives best solution quality; smaller β gives faster runtime (practically similar results).

LIP Applied to Knapsack and TSP



Knapsack ($d = 100$, $M = 100$ candidates): LIP outperforms plain Metropolis – informed proposals jump to higher-value neighbors.

TSP (13 cities):

- LIP ($M = 100$): evaluates 100 of $\binom{13}{2} = 78$ swaps
- Substantially faster convergence than plain Metropolis
- Winner in 9.66/100 runs vs. 0.33 for plain Metropolis

Cost: $\sim 100\times$ more expensive per step than Metropolis.

7.4 The Cross-Entropy Method – Overview

Different paradigm: Instead of walking on the state space, *update a family of distributions* $\{p_\theta\}$ so that it concentrates near the optimum.

CE Optimization Setup (Eq. 7.4.1–7.4.2)

$$s^{\mathrm{opt}} = \arg\max_{s \in \mathcal{S}} \pi_s = \arg\max_{s \in \mathcal{S}} h(s),$$
$$\gamma^{\mathrm{opt}} = \max_{s \in \mathcal{S}} h(s) \in \mathbb{R}.$$

- $h : \mathcal{S} \to \mathbb{R}$ – **performance function**
- $\{p_\theta : \theta \in \Theta\}$ – parametric family (our choice)
- θ_0 – initial parameter (e.g. uniform)

Core idea: relate the rare event $I(\gamma) = \mathbb{P}_{\theta_0}(h(\mathbf{X}) \geq \gamma)$ (probability of being near-optimal) to the estimation problem, then update θ to make this event less rare.

CE: Parametric Estimation Connection

Estimation problem:

$$l(\gamma) = \mathbb{P}_{\theta_0}(h(\mathbf{X}) \geq \gamma) = \sum_{s \in \mathcal{S}} \mathbf{1}(h(s) \geq \gamma)\, p_{\theta_0}(s) = \mathbb{E}_{\theta_0}\big[\mathbf{1}(h(\mathbf{X}) \geq \gamma)\big].$$

If $\gamma = \gamma^{\mathrm{opt}}$ and p_{θ_0} is uniform:

$$l(\gamma^{\mathrm{opt}}) = \frac{\#\{s : h(s) = \gamma^{\mathrm{opt}}\}}{|\mathcal{S}|} \approx \frac{1}{|\mathcal{S}|} \quad \text{(very small!)}.$$

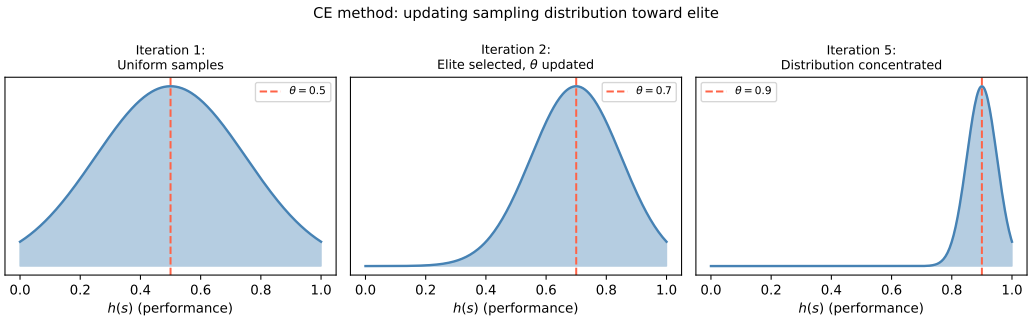
CE strategy: use IS-like parameter update to shift θ so that high-performance solutions are sampled more often.

CE Parameter Update (Eq. 7.4.5)

$$\theta' = \arg\max_{\theta} \frac{1}{M} \sum_{i:\, \mathbf{X}_i \in \mathcal{E}_t} \log p_{\theta}(\mathbf{X}_i),$$

where **elite set** $\mathcal{E}_t = \{\mathbf{X}_i : h(\mathbf{X}_i) \geq \gamma_t\}$.

CE Method: Schematic



Interpretation: At each iteration, samples are drawn from current p_θ. The top $(1 - \rho)$ fraction ("elite") are used to fit a new θ', shifting the distribution toward high-performance regions.

Algorithm 69: Cross-Entropy Method for Optimization I

Inputs: family $\{p_\theta\}$, θ_0, performance $h(\cdot)$, sample size M, elite fraction $\rho \in (0,1)$, smoothing $\alpha \in (0,1)$, stopping depth d (default 5).

Repeat: $t = 1, 2, \ldots$

1. **Update level γ_t:** Generate $\mathbf{X}_1, \ldots, \mathbf{X}_M \sim p_{\theta'_{t-1}}$. Sort performances $\eta_{(1)} \le \cdots \le \eta_{(M)}$.

$$\gamma_t = \eta_{(\lceil (1-\rho)M \rceil)}.$$

2. **Update parameter θ'_t:** from the *same* samples, let $\mathcal{E}_t = \{\mathbf{X}_i : h(\mathbf{X}_i) \ge \gamma_t\}$,

$$\theta' = \arg\max_\theta \frac{1}{M} \sum_{i=1}^{M} \mathbf{1}(h(\mathbf{X}_i) \ge \gamma_t) \log p_\theta(\mathbf{X}_i).$$

Apply **smoothed update:**

$$\theta'_t = \alpha\, \theta' + (1-\alpha)\, \theta'_{t-1}. \quad \text{(Eq. 7.4.6)}$$

Stop when $\gamma_t = \gamma_{t-1} = \cdots = \gamma_{t-d}$. **Return** elite set $\mathcal{E}$.

Algorithm 69: Cross-Entropy Method for Optimization II

Differences from CE for estimation:

- In estimation, θ_0 is *given*; in optimization, θ_0 is our choice (e.g. uniform).
- No likelihood ratio / IS weights needed (Eq. 7.4.5 uses $W \equiv 1$).
- The level γ_t *increases* over iterations until it converges to γ^{opt}.
- Output is the best *solution* found, not an estimate.

CE for the 0-1 Knapsack Problem

Parametric family: independent Bernoulli coordinates, $\theta = (\theta_1, \ldots, \theta_d)$, $\theta_i \in [0,1]$.

$$p_\theta(\mathbf{x}) = \prod_{j=1}^{d} \theta_j^{x_j} (1-\theta_j)^{1-x_j}.$$

Start with $\theta_0 = (1/2, \ldots, 1/2)$ (uniform).

CE Update for Knapsack (Eq. 7.4.7)

$$\theta'_k = \frac{\sum_{i=1}^{M} \mathbf{1}(h(\mathbf{X}_i) \geq \gamma_t)\, \mathbf{X}_i}{\sum_{i=1}^{M} \mathbf{1}(h(\mathbf{X}_i) \geq \gamma_t)}.$$

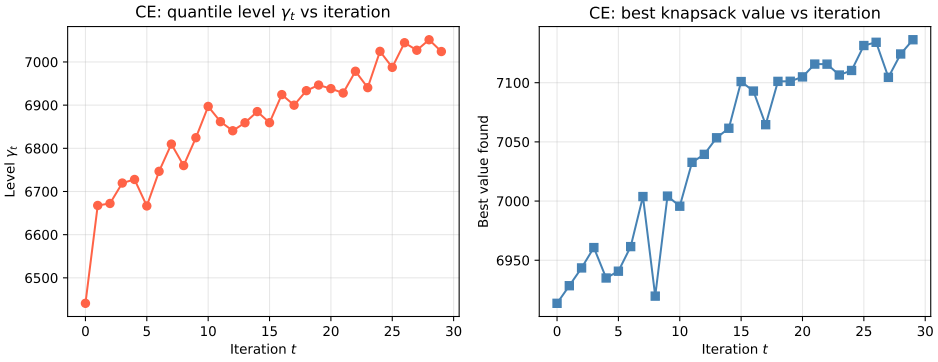
i.e. the **sample mean of elite solutions** – a simple average!

The k-th coordinate of θ' is the fraction of elite samples that include item k.

CE Knapsack: Python Implementation

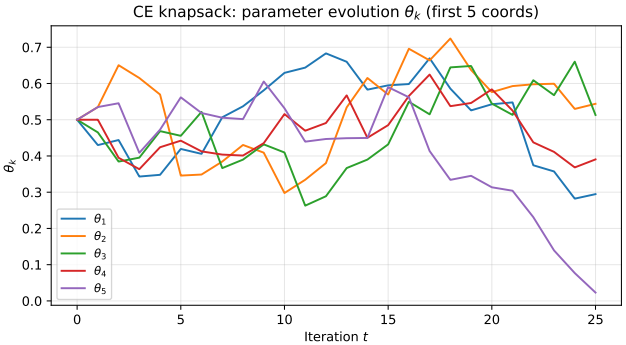
```python
import numpy as np

def ce_knapsack(d, w, v, W, theta0, M=200, rho=0.1, alpha=0.7,
                d_stop=5, max_iter=50):
    theta = theta0.copy()
    gammas, best_vals = [], []
    gamma_history = []
    for t in range(max_iter):
        # Step 1: sample and compute performance
        X = (np.random.rand(M, d) < theta).astype(float)
        h = np.array([x @ v if x @ w <= W else 0.0 for x in X])
        # Step 2: compute elite level
        gamma_t = np.quantile(h, 1 - rho)
        gammas.append(gamma_t)
        # Step 3: elite update
        elite = h >= gamma_t
        if elite.sum() > 0:
            theta_new = X[elite].mean(axis=0)
            theta = alpha * theta_new + (1 - alpha) * theta
        best_vals.append(h.max())
        gamma_history.append(gamma_t)
        # Stopping criterion
        if (len(gamma_history) > d_stop and
                len(set(gamma_history[-d_stop:])) == 1):
            break
    return theta, best_vals, gammas
```

CE Convergence: Knapsack Results



Left: the quantile level γ_t increases monotonically, converging to γ^{opt}. **Right:** best value found improves rapidly and plateaus. CE converges in ~ 30 iterations even for $d = 100$.

CE Parameter Evolution



Each θ_k converges to 0 or 1:

- $\theta_k \to 1$: item k is always in elite solutions (include it)
- $\theta_k \to 0$: item k is never in elite solutions (exclude it)

Items with high value-to-weight ratio converge to $\theta_k = 1$ faster.

Smoothing parameter α prevents premature convergence.

CE for the TSP: Parametric Family

Parametric family for permutations: $\theta = (\theta_{ij})$ – a stochastic $n \times n$ matrix with $\theta_{ii} = 0$, $\theta_{ij} \geq 0$, $\sum_j \theta_{ij} = 1$.

Algorithm 70: Sample Permutation from θ

1. Set $\sigma_1 = 1$ (start city).
2. For $k = 1, \ldots, n-1$:
   - Take row k of θ; zero out already-visited cities.
   - Normalize remaining entries to get probability vector $\mathbf{q}^{k+1}$.
   - Sample σ_{k+1} from $\mathbf{q}^{k+1}$.

CE Update for TSP (Eq. 7.4.9)

$$\theta'_{ab} = \frac{\sum_{i=1}^{M} \mathbf{1}(h(\mathbf{X}_i) \geq \gamma_t) \sum_{k=1}^{n-1} \mathbf{1}(\sigma_k^{(i)} = a,\, \sigma_{k+1}^{(i)} = b)}{\lambda_a},$$

where λ_a is a normalising constant ensuring row sums equal 1.

CE for TSP: Python Skeleton

```python
import numpy as np

def sample_permutation(theta, n, rng):
    """Algorithm 70: sample a tour from stochastic matrix theta."""
    sigma = [0]    # start at city 0
    for k in range(n - 1):
        row = theta[sigma[-1]].copy()
        for already in sigma:
            row[already] = 0.0           # exclude visited
        row /= row.sum()                 # normalise
        next_city = rng.choice(n, p=row)
        sigma.append(next_city)
    return sigma

def ce_tsp_update(elite_tours, n):
    """Compute theta' from elite tours (Eq. 7.4.9)."""
    counts = np.zeros((n, n))
    for tour in elite_tours:
        for k in range(len(tour) - 1):
            counts[tour[k], tour[k+1]] += 1
    # Normalise each row
    row_sums = counts.sum(axis=1, keepdims=True)
    row_sums[row_sums == 0] = 1
    return counts / row_sums
```

7.5 Metropolis–CE: A Hybrid Approach

Metropolis: random walk, state = current solution.
CE: state = distribution p_θ; M solutions sampled each step.

Hybrid idea (Algorithm 72): at each iteration,

- Generate M *neighbors* of $\mathbf{X}^{(t-1)}$ from the transition distribution $\mathbf{Q}_{\theta'_{t-1}}(\mathbf{X}^{(t-1)}, \cdot)$.
- Update θ as in CE using elite samples.
- Set next state $\mathbf{X}^{(t)}$ = best sample.

Metropolis-CE Parameter Update (Eq. 7.5.1)

$$\theta' = \arg\max_{\theta} \frac{1}{M} \sum_{i=1}^{M} \mathbf{1}(h(\mathbf{X}_i) \geq \gamma_t) \log \mathbf{Q}_\theta(\mathbf{X}^{(t-1)}, \mathbf{X}_i).$$

Smoothed: $\theta'_t = \alpha\, \theta' + (1-\alpha)\, \theta'_{t-1}$.

Metropolis-CE for Knapsack

Distribution family: $\theta = (\theta_1, \ldots, \theta_d)$, where $\sum_k \theta_k = 1$, $\theta_k \geq 0$. Neighbor $\mathbf{X}'$ is formed by picking coordinate k to flip with probability θ_k.

CE Update for Knapsack Neighbors (Eq. 7.5 detail)

$$\theta'_k = \frac{C_k}{\sum_{j=1}^{d} C_j}, \qquad C_k = \sum_{i=1}^{M} \mathbf{1}(h(\mathbf{X}_i) \geq \gamma_t).$$

C_k counts how often coordinate k was flipped in elite samples.

Result (100 runs): Metropolis-CE *always* wins among competing algorithms with $\rho = 0.3$, $M = 30$, $\alpha = 0.001$, $T = 100$ steps.

Metropolis-CE for TSP

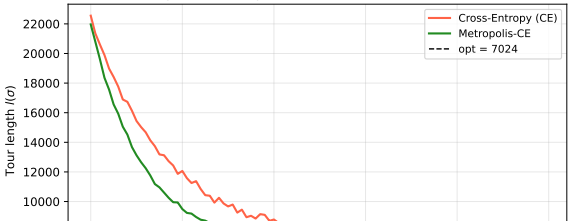
Distribution family for swaps: $\theta = (\theta(i,j))_{i<j}$, where $\theta(i,j)$ is the probability of swapping positions i and j.

CE Update for TSP Swaps (Eq. 7.5.2)

$$\theta'(a,b) = \frac{C_{(a,b)}}{\sum_{i<j} C_{(i,j)}}, \qquad C_{(a,b)} = \sum_{i=1}^{M} \mathbf{1}(h(\mathbf{X}_i) \geq \gamma_t)\, \mathbf{1}((a,b) \text{ swapped in } \mathbf{X}_i).$$

Simulation ($M = 100$, $\rho = 0.35$, $\alpha = 0.7$): In 61 out of 100 runs the CE method (Alg. 71) won; in 29/100 runs Metropolis-CE won; LIP won 9.66/100 times.

Fig 7.5 – Metropolis-CE vs CE on TSP (13 cities)



Algorithm 72: Metropolis-CE (Python Sketch)

```
def metropolis_ce(x0, h, sample_neighbors, update_theta,
                  theta0, T=100, M=30, rho=0.3, alpha=0.001):
    """Metropolis-CE hybrid (Algorithm 72)."""
    theta = theta0.copy()
    X_cur = x0.copy()
    best = X_cur.copy()
    for t in range(1, T + 1):
        # Generate M neighbors of X_cur using distribution Q_theta
        Xs = [sample_neighbors(X_cur, theta) for _ in range(M)]
        perfs = np.array([h(X) for X in Xs])
        # Compute elite level
        gamma_t = np.quantile(perfs, 1 - rho)
        elite_mask = perfs >= gamma_t
        # Update theta from elite neighbors
        theta_new = update_theta(Xs, elite_mask, theta)
        theta = alpha * theta_new + (1 - alpha) * theta
        # Move to best sample
        best_idx = np.argmax(perfs)
        X_cur = Xs[best_idx]
        if h(X_cur) > h(best):
            best = X_cur.copy()
    return best
```

7.6 Summary of Simulation Results

All algorithms run 100 times, each for a fixed number of steps.

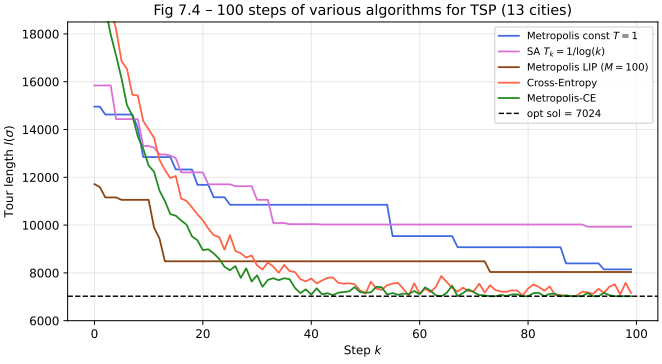
7.6.1 Travelling Salesman Problem (13 cities)

- Optimal solution: **7024** (Held-Karp, $O(n^2 2^n)$ time)
- All experiments use distance matrix **M** from Fig. 7.3

Algorithm	Win score	Best (median)	Time (s)
Metropolis ($T = 1$)	0.33	~ 8200	< 1
SA ($T_k = 1/\log k$)	0.00	~ 7800	< 1
LIP ($M = 100$)	9.66	~ 7500	~ 10
Cross-Entropy	61.0	~ 7300	~ 230
Metropolis-CE	29.0	~ 7050	~ 10

CE wins most often on TSP; Metropolis-CE is a faster alternative.

TSP Results: Visual Comparison

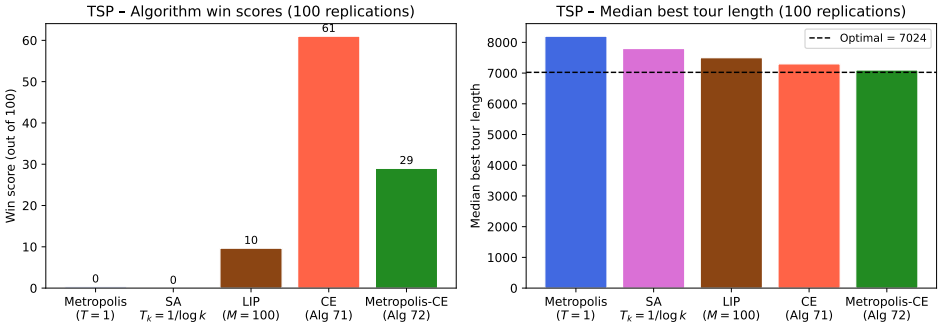


Key observations:

- Plain Metropolis ($T = 1$) gets trapped at high tour lengths
- SA improves more quickly early on
- CE and Metropolis-CE converge fastest to near-optimal
- LIP provides good improvement at moderate cost

Dashed line: optimal = 7024.

Algorithm Comparison: Win Scores



Win score = fraction of 100 replications where an algorithm found the best tour length. CE-based methods dominate for TSP.

7.6.2 Knapsack Problem Results

Setup: $d = 100$, $w_i = i$, $v_i = i^{1.2}$, $W = 3000$.

Summary Table 7.3 (100 replications)

Algorithm	Best value (median)	Win score
Metropolis ($T = 1$)	~ 6800	low
Gibbs sampler ($T = 1$)	~ 6800	low
Metropolis LIP	~ 7100	moderate
Cross-Entropy	~ 7200	high
Metropolis-CE	~ 7400	**highest**

Observations:

- Metropolis-CE always wins for knapsack in 100-step experiments
- CE and LIP both superior to uninformed Metropolis
- For knapsack the optimal value is not known (NP-hard), but CE-based methods reliably reach $\mathbf{v} \cdot \mathbf{x} \approx 7400$

7.7 Bibliographical Notes

- **Metropolis for decryption:** Diaconis [24] – early application of MCMC to code-breaking.
- **Simulated Annealing:** Hajek [76] proved convergence under logarithmic cooling. Kirkpatrick, Gelatt, Vecchi (1983) introduced SA for combinatorial optimisation.
- **Locally Informed Proposals:** Grathwohl et al. [71] "Oops I took a gradient: Scalable sampling for discrete distributions" – gradient-based approximations to LIP.
- **Cross-Entropy method:** Rubinstein & Kroese [31] – comprehensive reference. FACE algorithm = fully automated cross-entropy [31, Alg. 4.1].
- **TSP:** Held-Karp algorithm [82] optimal solution in $O(n^2 2^n)$. Optimal for 13-city instance found in 0.068 s.
- Convergence theory of non-homogeneous Markov chains follows from Proposition 6.3.2 (MH stationarity).

7.8 Selected Exercises I

Exercise 7.L.1 (Substitution Cipher)

Implement the Metropolis-based decryption algorithm. There are several helper files: `ch7_cipher_*`. Run for 3000 steps starting from a random permutation. Report the evolution of log-likelihood and the final decrypted message.

Exercise 7.L.2 (Permutation Distribution)

Consider $\pi_\sigma = \frac{1}{z}\lambda^{d(\sigma,\sigma_{\mathrm{id}})}$, $\lambda > 0$, where $d(\sigma,\sigma') = \frac{1}{n}\sum_i |\sigma(i) - \sigma'(i)|$.

(a) Implement the Metropolis algorithm for this distribution using the TSP proposal (Algorithm 66). For which λ is it applicable?

(b) Repeat with the Kendall τ distance.

(c) Use the 2-opt proposal (Remark 7.2.4) and compare.

7.8 Selected Exercises II

Exercise 7.L.4 (Knapsack, Smaller Capacities)

Re-implement the Gibbs sampler for knapsack with $W \in \{50, 500, 1000, 3000\}$. Compare with random sampling.

Exercise 7.L.5 (SA for Knapsack)

Construct a simulated annealing version of the knapsack Gibbs sampler with $\pi_{\mathbf{x}}^{T_k} = \exp(\mathbf{v} \cdot \mathbf{x}/T_k)/z_{T_k}$ for $\mathbf{w} \cdot \mathbf{x} \leq W$. Compare:

- Fixed $T_k = 1$ (no annealing)
- Logarithmic $T_k = 1/\log(k)$

for $W = 3000$ and $W = 50$.

Exercise 7.L.6 (CE for TSP, Alternative Parametrisation)

Implement CE for TSP with a vector $\theta = (\theta_1, \ldots, \theta_n)$ over cities (instead of a stochastic matrix). Compare convergence speed and solution quality with the matrix parametrisation.

Chapter 7 Summary

Five Stochastic Optimization Methods

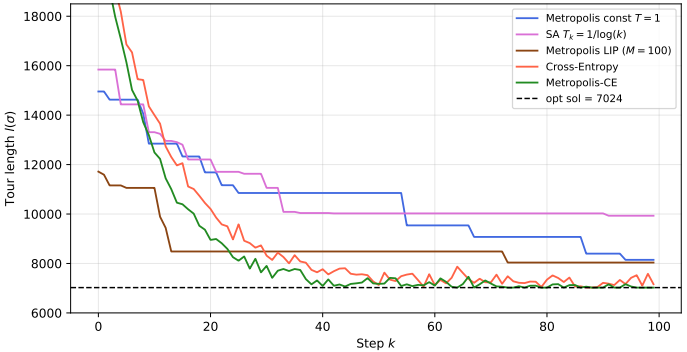
Method	State	Key idea	Tuning
Metropolis (T fixed)	solution	Boltzmann dist.	T, graph
Simulated Annealing	solution	decrease T_k	cooling schedule
Locally Informed	solution	informed proposal	β, $g(\cdot)$
Cross-Entropy	distribution θ	update elite	M, ρ, α
Metropolis-CE	sol. + dist.	hybrid walk	all of above

Key takeaways:

- All methods use the Boltzmann distribution as a bridge between probability and optimisation.
- Simulated annealing converges to the global optimum if cooling is slow enough ($\sum e^{-c/T_j} = \infty$).
- CE-based methods often outperform Metropolis-based ones on combinatorial problems (TSP, knapsack).
- Metropolis-CE combines best of both: cheap per-step cost with CE-quality guidance.
- Choice of method depends on problem size, computational budget, and available gradient/structure information.

Comparison: All Algorithms on TSP

Fig 7.4 – 100 steps of various algorithms for TSP (13 cities)



Method	Character
Metropolis (const. T)	Random walk, can get trapped
Simulated Annealing	Escapes traps via high-T phase
LIP	Informed walk, moderate cost
Cross-Entropy	Distribution-level learning
Metropolis-CE	Hybrid: best of both worlds